Yolov11 Garbage Recognition and Sorting

1. Function Description

After the program runs, the robotic arm will move to its initial position, then place wooden blocks with garbage label codes within the camera's view. Press the spacebar, and the robotic arm will grasp based on the recognized garbage label code results and place them at designated positions according to the garbage type.

2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start Yolov11 garbage recognition node
ros2 run dofbot_yolov11 yolov11_garbage
# Start grasping node
ros2 run dofbot_sorting grasp
```

After successfully running the program, the robotic arm will move to its initial position, then place wooden blocks with garbage label codes within the camera's view as shown in the figure below.

Press the spacebar to start sorting the recognized garbage label codes.

3. Code Analysis

3.1. yolov11_garbage

After entering the docker container, the source code path is located at:

```
/root/dofbot_sorting/dofbot_sorting/color_sorting.py
```

Import necessary library files:

```
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image
from std_msgs.msg import  Bool
from cv_bridge import CvBridge
import cv2 as cv
import threading
from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
import time
import math
from Arm_Lib import Arm_Device
from ultralytics import YOLO
```

Load model:

```
model = YOLO("/root/ultralytics/ultralytics/best.onnx")
```

Create robotic arm control object and control the robotic arm to move to initial position:

```
self.Arm = Arm_Device()
self.init_joints = [90, 120, 0, 0, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Define four types of garbage lists:

```
self.recyclable_waste=['Newspaper','Zip_top_can','Book','Old_school_bag']
self.toxic_waste=
['Syringe','Expired_cosmetics','Used_batteries','Expired_tablets']
self.wet_waste=['Fish_bone','Egg_shell','Apple_core','Watermelon_rind']
self.dry_waste=
['Toilet_paper','Peach_pit','Cigarette_butts','Disposable_chopsticks']
```

Create publishers and subscribers:

```
# Create publisher to publish calculated joint 5 angle reference value topic
messages
self.TargetJoint5_pub = self.create_publisher(Int16, "set_joint5", 10)
# Create publisher to publish recognized position information topic messages
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)
# Create subscriber to subscribe to grasp completion topic messages
self.subscription =
self.create_subscription(Bool,'grasp_done',self.GraspStatusCallback,qos_profile=
1)
# Create subscriber to subscribe to image topic messages
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
```

Grasp completion topic callback function GraspStatusCallback:

```
def GraspStatusCallback(self,msg):
    print("**")
    if msg.data == True:
        self.pubPos_flag = True
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Process received image topic messages, convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')
    result_image = np.copy(rgb_image)
    frame = np.copy(rgb_image)
    # Call yolov11 model, pass image for recognition, store recognition results
in results
    results = model(frame)
    key = cv2.waitKey(10)
    # If spacebar is pressed, it means garbage label code position information
topic can be published
    if key == 32:
        self.pubPos_flag = True
    # Extract recognition results
    boxes = results[0].boxes
    # If recognition results are not empty, it means garbage on the garbage label
code was recognized
    if boxes != [None]:
        # Traverse recognition results
        for box in boxes:  # detections per image
            # Extract top-left and bottom-right recognition boxes of garbage
label code
            x_min, y_min, x_max, y_max = map(int, box.xyxy[0])
            class_id = int(box.cls)
            # Extract confidence
            confidence = float(box.conf)
            label = f"{model.names[class_id]} {confidence:.2f}"
            print("model.names[class_id]: ",model.names[class_id])
            # Calculate center point coordinates
            center_x = (x_min + x_max) // 2
            center_y = (y_min + y_max) // 2
```

```
            # Based on center point coordinates, convert coordinates to
normalized coefficients a and b. a is x-axis offset coefficient, b is y-axis
offset coefficient. The calculated ba values are the center point coordinates in
the world coordinate system, which will eventually be sent to the sorting grasp
node.
            (a, b) = (round(((320 - center_x) / 4000), 5), round(((480 -
center_y) / 3000) * 0.8+0.15, 5))
            # If self.pubPos_flag value is True, it means garbage label code
position information topic can be published
            if self.pubPos_flag == True:
                # self.pubPos_flag value is False, indicating position topic
publishing is prohibited to prevent sending recognized position messages again
during grasping process
                self.pubPos_flag = False
                # Create position information topic message object
                center = AprilTagInfo()
                center.x = b
                center.y = a
                center.z = 0.03
                # Determine which category the garbage belongs to, then assign
value to center.id
                if str(model.names[class_id]) in self.recyclable_waste:
                    center.id = 1
                elif str(model.names[class_id]) in self.toxic_waste:
                    center.id = 2
                elif str(model.names[class_id]) in self.wet_waste:
                    center.id = 3
                elif str(model.names[class_id]) in self.dry_waste:
                    center.id = 4
                print("center: ",center)
                # Publish garbage label code position information topic
                self.pos_info_pub.publish(center)
    annotated_frame = results[0].plot()
    annotated_frame = cv2.cvtColor(annotated_frame, cv2.COLOR_RGB2BGR)
    cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))
    key = cv2.waitKey(1)
```

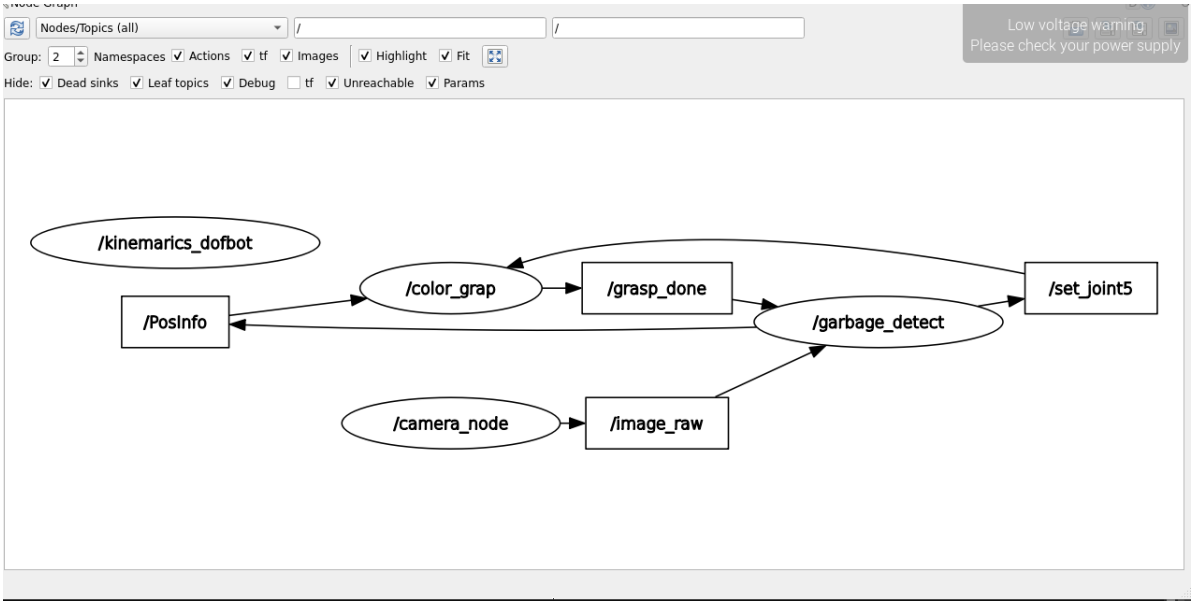
3.2. grasp

Already analyzed in [13.3D Sorting and Grasping Course]-[1. AprilTag ID Sorting]-[3. Code Analysis], you can refer to the content there.

4. Node Communication Diagram

After entering the docker container, enter the following commands in the docker terminal:

```
ros2 run rqt_graph rqt_graph
```


Nodes/Topics (all)
Group: 2 Namespaces Actions tf Images Highlight Fit
Hide: Dead sinks Leaf topics Debug tf Unreachable Params
Low voltage warning
Please check your power supply
/kinemarics_dofbot
/color_grap
/grasp_done
/set_joint5
/PosInfo
/garbage_detect
/camera_node
/image_raw